

Oral Projet Programmation pour la Finance

Partie Business :

Bonjour à tous. Quand on pense aux marchés financiers, l'image qui vient souvent à l'esprit est celle d'un environnement fondamentalement directionnel. On achète une action en espérant que son prix monte, et on panique dès que le marché s'effondre. C'est le modèle classique : on parie sur l'avenir. Mais que se passe-t-il quand l'avenir devient totalement illisible ? Quand une crise géopolitique ou économique éclate et que tous les indices plongent dans le rouge ?

Pour les investisseurs institutionnels, qu'il s'agisse d'une trésorerie de grand groupe ou d'un fonds de pension, subir une perte de 30 % sur un portefeuille n'est pas une option. Ces acteurs ne cherchent pas le pari spéculatif du siècle, ils cherchent la **stabilité absolue**. Ils ont besoin de générer du rendement de manière constante, que le marché monte, stagne, ou s'effondre.

C'est ici qu'intervient la finance quantitative, et plus précisément l'architecture logicielle que nous avons développée pour ce projet : une plateforme d'**Arbitrage Statistique**, exploitant la stratégie du *Pairs Trading*.

Le concept d'origine est né dans les années 80 sur les bureaux de trading quantitatif de Wall Street. L'idée business est brillante de simplicité : au lieu de parier sur la direction d'une action isolée, on va parier sur la **relation** entre deux actions concurrentes.

Prenons un exemple concret avec Coca-Cola et PepsiCo. Ces deux géants évoluent dans le même secteur, subissent les mêmes coûts de matières premières et font face aux mêmes régulations. Historiquement, leurs cours de bourse dansent ensemble. Mais parfois, à cause du bruit du marché ou d'une panique temporaire, cette danse se désynchronise. Coca-Cola monte un peu trop haut, PepsiCo baisse un peu trop bas. L'élastique invisible qui les relie se tend.

Notre algorithme ne se demande pas si le secteur des sodas va globalement monter ou baisser le mois prochain. Il constate simplement que l'élastique est trop tendu et qu'il va mathématiquement finir par claquer, ramenant les deux prix vers leur équilibre historique.

Pour exploiter cette anomalie, le moteur que nous avons codé construit un portefeuille dit **Market-Neutral** (neutre au marché). Concrètement, lorsque l'élastique se tend, nous achetons l'action sous-évaluée et nous vendons simultanément à découvert l'action surévaluée.

Pourquoi est-ce si puissant d'un point de vue commercial ? Parce que si le marché mondial s'effondre le lendemain matin, notre action achetée perdra de la valeur, mais notre action vendue à découvert en gagnera tout autant. Le risque directionnel est instantanément annulé. Notre seul moteur de performance, notre unique création de valeur, c'est le resserrement de l'écart entre Coca et Pepsi.

Notre solution s'adresse donc directement à une clientèle institutionnelle exigeante, cherchant à diversifier ses revenus avec une stratégie totalement décorrélée des krachs boursiers.

C'est ce qui nous amène à la partie centrale de notre projet : comment prouver que cet "élastique" existe vraiment sans se faire piéger par de simples illusions statistiques, et comment gérer l'instabilité permanente du marché ?

Partie Problèmes et Solutions :

La première erreur que font beaucoup de traders débutants, c'est de chercher deux actions qui se ressemblent visuellement sur un graphique, ou qui ont une forte corrélation. Le problème, c'est que la corrélation en finance est souvent une illusion dangereuse. Deux actions peuvent monter ensemble pendant cinq ans simplement parce que le marché global monte, mais elles n'ont aucun lien fondamental entre elles. Dès qu'une crise éclate, elles partent dans des directions opposées, et notre portefeuille s'effondre.

Pour éviter ce piège, nous n'utilisons pas la corrélation, mais un concept statistique beaucoup plus robuste qui a valu le prix Nobel d'économie à ses créateurs : **la cointégration**. Pour vulgariser, imaginez un ivrogne qui promène son chien sans laisse. L'ivrogne avance de manière totalement imprévisible, et le chien court dans tous les sens. Individuellement, leurs trajectoires sont ce qu'on appelle en mathématiques des "marches aléatoires". Il est impossible de prédire où sera l'homme, ou où sera le chien dans dix minutes. Mais, s'ils sont liés par une laisse invisible – la cointégration – on sait avec une certitude mathématique que la distance entre les deux ne dépassera jamais la longueur de la laisse. S'ils s'écartent trop, ils finiront par se rapprocher.

Pour prouver que cette laisse existe entre deux actions, nous avons implémenté la méthode d'Engle-Granger en deux étapes. D'abord, notre algorithme effectue une régression linéaire pour trouver la taille de la laisse, ce qu'on appelle le *Hedge Ratio*. Ensuite, il extrait l'écart entre les deux prix – le fameux spread – et lui fait passer un test statistique très strict : le test de **Dickey-Fuller Augmenté**, ou ADF.

Ce test vérifie une hypothèse simple : est-ce que cet écart a tendance à revenir vers zéro, ou est-ce qu'il part à l'infini ? Si le test réussit, bingo : la série est dite "stationnaire". Nous avons la preuve mathématique que nos actions sont cointégrées.

Mais, et c'est là que notre projet devient vraiment intéressant, nous avons vite frappé un mur. En testant ce modèle classique sur les marchés réels des quatre dernières années, notre algorithme ne gagnait pas d'argent. Pourquoi ? À cause d'un problème majeur : le ratio de couverture statique.

Le monde change. Entre 2020 et aujourd'hui, nous avons eu le COVID, le retour de l'inflation, et des chocs sur les taux d'intérêt. Si on calcule notre *Hedge Ratio* – notre fameuse laisse – comme une simple moyenne fixe sur quatre ans, on part du principe que la relation entre nos deux entreprises est immuable. C'est faux. L'élastique se déforme avec le temps. Si notre algorithme utilise un repère obsolète, il va détecter de faux signaux d'achat et de vente, et accumuler les pertes.

Pour résoudre ce problème complexe, nous avons dû abandonner les régressions classiques pour implémenter un algorithme de pointe : le **Filtre de Kalman**. Le Filtre de Kalman ne voit pas la relation entre nos actions comme un chiffre fixe, mais comme une variable vivante, qui évolue chaque jour. À chaque nouvelle seconde de cotation, le filtre compare le prix réel du marché avec sa propre prédiction. S'il s'est trompé, il corrige instantanément son ratio de couverture. Il s'adapte en temps réel aux changements de régime macroéconomique. C'est le GPS de notre algorithme. Grâce à lui, notre "laisse" mathématique ne casse jamais, elle s'ajuste dynamiquement à la réalité du marché.

Il restait cependant un dernier obstacle, de loin le plus redoutable : le facteur Temps. En finance institutionnelle, le temps, c'est littéralement de l'argent. Quand nous vendons à découvert l'action qui est trop chère, nous devons l'emprunter à un courtier. Et ce courtier nous facture un taux d'intérêt chaque jour où nous gardons cette position ouverte.

Si l'élastique entre nos deux actions met 6 mois à se resserrer, le gain que l'on fera à la fin sera totalement effacé par les frais d'emprunt que nous aurons payés pendant 180 jours. C'est ce qui tue 90% des modèles théoriques lorsqu'ils passent en production.

Notre solution ? Modéliser la dynamique de notre écart de prix à travers une équation différentielle stochastique appelée le **processus d'Ornstein-Uhlenbeck**. Pour faire simple, cette équation nous permet de calculer à l'avance la "demi-vie" de notre spread. C'est-à-dire le temps exact qu'il faudra pour que l'écart se résorbe de moitié. Si l'équation nous dit que la demi-vie est de 45 jours, notre algorithme lève le drapeau rouge : le retour à la normale est trop lent, les frais d'emprunt vont nous ruiner, on n'investit pas.

En revanche, grâce au Filtre de Kalman qui absorbe la tendance à long terme, la demi-vie de nos opportunités est souvent réduite à moins de 24 heures. Nous entrons, nous captons l'anomalie de prix, et nous sortons avant même que les frais structurels n'aient le temps de ronger notre profit.

Ainsi, en combinant la Cointégration pour prouver la relation, le Filtre de Kalman pour s'adapter aux crises, et le processus d'Ornstein-Uhlenbeck pour maîtriser les coûts de friction, nous avons transformé une simple idée statistique en un véritable moteur de trading institutionnel.

Partie Code :

Parlons maintenant du code. Pour garantir une robustesse de niveau industriel, nous avons structuré notre backend Python autour du principe de séparation des responsabilités. Le projet est divisé en quatre moteurs distincts : un module d'ingestion de données (MarketData), un moteur statistique pour les tests de stationnarité (AdvancedStatTests), le filtre dynamique (KalmanFilterStatArb), et enfin, le simulateur de salle des marchés (InstitutionalPortfolioSizer). Le tout est ensuite branché sur une interface visuelle Streamlit, qui reste complètement "aveugle" et ne fait aucun calcul.

Si je devais isoler la ligne de code la plus importante de tout ce projet, ce ne serait pas une équation complexe. Ce serait cette ligne très simple dans notre backtester : `df['position'] = df['target_position'].shift(1)`.

En finance quantitative, l'ennemi public numéro un s'appelle le *Lookahead Bias*, ou le biais d'anticipation. Imaginons que notre Z-score franchisse notre seuil d'achat à 16h00, juste à la clôture du marché. Si mon algorithme calcule son profit en utilisant la variation de prix de la journée qui vient de s'écouler, il triche. Il utilise une information du futur. Cette fonction `.shift(1)` décale temporellement toute notre matrice de données d'un cran. Elle garantit de manière stricte que le signal généré à la clôture du jour J ne sera exécuté et détenu que pour le jour J+1. Sans ce décalage d'index, un backtest peut afficher 200% de rentabilité tout en étant totalement faux.

Une fois que notre gestion du temps est infaillible, nous pouvons regarder le cœur mathématique de notre algorithme : la boucle d'état du Filtre de Kalman.

En Python, et particulièrement avec la bibliothèque Pandas, la règle d'or est de toujours "vectoriser" ses calculs, c'est-à-dire de traiter toute la colonne de données d'un seul coup pour des raisons de performance. Pourtant, dans notre fonction `calculate_dynamic_beta`, nous utilisons une boucle `for` classique : `for t in range(1, n):`.

C'est une obligation mathématique. Le Filtre de Kalman est basé sur une chaîne de Markov. Cela signifie que l'état de notre ratio de couverture à l'instant T dépend *strictement* de son état à l'instant T-1 et de la nouvelle erreur d'observation, qu'on appelle l'innovation. On ne peut pas calculer le futur sans avoir résolu le présent. Dans cette boucle, à chaque itération, le code calcule le Gain de Kalman (K). Si le marché est extrêmement bruité, ce gain se rapproche de zéro, et notre algorithme ignore l'anomalie de prix. Si le signal est fort, le gain augmente et met à jour notre équation de couverture. C'est ce bloc de code précis qui donne à notre modèle sa capacité d'adaptation en temps réel.

Enfin, la dernière mécanique vitale de notre code se trouve dans l'allocation du capital. C'est le fameux *Position Sizing*.

Puisque notre Filtre de Kalman recalcule le ratio de couverture tous les jours, notre cible de couverture évolue en permanence. Si, le lundi, l'algorithme nous dit de vendre 31 actions Pepsi pour nous couvrir, et que le mardi il nous dit d'en vendre 31,5... la tentation serait de racheter ou de revendre des fractions d'actions tous les jours pour coller parfaitement à la mathématique.

Dans le code, nous avons formellement bloqué cela en utilisant une fonction `np.where` conditionnée par le changement de signal, couplée à un `.ffill()` (Forward Fill). Pourquoi avoir codé ce verrouillage ? Pour empêcher ce qu'on appelle le *Churning*, la sur-rotation du portefeuille. Si notre code passait des micro-ordres tous les jours pour ajuster le ratio, les 5 points de base de commissions de courtage nous ruineraient en moins d'une semaine. Notre algorithme calcule la taille exacte de la position au moment de l'entrée, la verrouille dans la matrice de données, et encaisse les variations jusqu'à la sortie, protégeant ainsi notre profit brut des frictions du marché.

Partie Finale (Exemple + Conclusion) :

Pour conclure cette présentation, prenons un peu de recul et regardons le résultat final sur un cas d'usage concret. Nous avons fait tourner notre moteur d'arbitrage sur deux géants bancaires américains : JPMorgan et Bank of America, sur les quatre dernières années, avec un Z-score d'entrée très strict de 2.5.

Sur le papier, le résultat net net affiché par notre tableau de bord est un Sharpe Ratio de 0.30 et un Max Drawdown de -8.7 %. À première vue, face aux rendements insolents que l'on voit parfois sur les réseaux sociaux ou dans des projets académiques survendus, cela peut paraître modeste. Mais dans la réalité des salles des marchés, c'est exactement ce profil de rendement qui est recherché et financé.

Pourquoi ? Parce que ce chiffre est authentique. Il a survécu à 5 points de base de commissions de courtage, au slippage de l'exécution, et surtout, aux frais d'emprunt payés pour la vente à découvert. C'est un rendement absolu, totalement décorrélé des fluctuations du marché global. Une banque d'investissement ne va jamais trader cette paire de manière isolée. Elle va déployer cet algorithme simultanément sur une centaine de paires d'actions cointégrées et y appliquer un effet de levier. Grâce à la loi des grands nombres et à la mutualisation du risque, un Sharpe de 0.30 sur une multitude d'actifs se transforme mathématiquement en un produit financier extrêmement performant et sécurisé.

Avant de terminer, j'aimerais prendre une minute pour mettre en perspective nos choix mathématiques face aux grands standards de la finance, comme la théorie de Markowitz ou le modèle de Black-Scholes.

Il est vrai qu'utiliser un Filtre de Kalman pour mettre à jour notre ratio de couverture expose au risque de sur-paramétrage. Sans une calibration rigoureuse, cela peut ressembler à une "régression de régression" qui chasserait le bruit du marché. C'est pourquoi la prochaine étape industrielle consistera à calibrer les matrices de covariance du filtre par la méthode du Maximum de Vraisemblance, afin de garantir une robustesse statistique absolue.

Cependant, il était mathématiquement impossible d'utiliser Markowitz ou Black-Scholes pour notre moteur. Markowitz optimise un portefeuille d'actifs qui suivent des marches aléatoires sur le long terme. Notre projet fait exactement l'inverse : nous utilisons la cointégration pour isoler un spread qui n'est *justement pas* une marche aléatoire, mais un processus stationnaire.

De même, bien que notre stratégie Market-Neutral s'apparente dans l'esprit à la vente d'une option sur la volatilité du spread — on encaisse de petits gains très probables en pariant que l'élastique ne va pas casser —, notre portefeuille au comptant reste un produit au rendement strictement linéaire. Nous n'avons ni la convexité du Gamma, ni la date d'expiration d'une option pricée par Black-Scholes

Bien sûr, notre plateforme n'est qu'une première itération et les perspectives d'amélioration pour atteindre un standard industriel sont nombreuses.

La première évolution naturelle serait d'hybrider ce modèle statistique avec du Machine Learning. Actuellement, notre algorithme achète aveuglément dès que le Z-Score touche 2.5. Nous pourrions entraîner une forêt aléatoire (Random Forest) ou un réseau de neurones à analyser les carnets d'ordres profonds au moment exact de ce franchissement, pour filtrer les faux signaux et augmenter drastiquement notre *Win Rate*.

Ensuite, pour absorber des milliers de ticks par seconde sur des centaines de paires, il faudrait migrer ce code Python vers un langage compilé de bas niveau comme le C++ ou Rust pour la partie exécution, et déployer l'architecture sur des serveurs cloud à très faible latence, au plus près des places boursières.